Class Cookies

A Projector-First Platform of Classroom Games and Tools for English Teachers

Technical report

Jacobus Barnard

Seoul, South Korea

October 2026

The author is the product owner and designer of Class Cookies. The software it describes was written by AI coding agents (Claude Code) under his direction; Chapter 5 explains how.

ABSTRACT

Class Cookies is a platform of classroom games and tools for English teachers, built for a single projector or TV and no student devices. It grew out of 59 Seconds, a speaking game I built as my MSIT capstone in the summer of 2026, and now joins five games (59 Seconds, Taco Trivia, Mango Mafia, Super Mushroom Man and Cookie Code), eight free classroom tools, a classroom manager, a worksheet library and a teachers' forum under one sign-in and one subscription. Each game is a separate application on its own subdomain; they share a session cookie and a single database function that answers whether a teacher has subscribed, and row-level security on every one of the platform's 25 tables decides who may read or change each row.

I am the product owner and designer. The code was written by AI coding agents: 59 Seconds began as a Lovable scaffold, and Claude Code has written the code since, in small steps that I approve, checked by tests I set and by two chat-based review agents. On the six production branches, 694 of 869 commits carry a Claude co-author line. This report describes that method as it was practised, including what went wrong.

Measured on 7 October 2026, 3,783 automated tests passed across five of the six codebases (Taco Trivia has none), five of the six built cleanly from fresh clones and the sixth built on its host, and seven of eleven public pages scored at least 93 for Lighthouse Performance on both desktop and mobile. Three of the other four are slowed by large images, and the fourth by fonts and styles that block the first paint on mobile. On 8 and 9 October 2026 I made a real test payment on my own card, which took the billing path through payment, access in every paid area, cancellation, a switch to the yearly plan and refunds; it found three faults, all fixed by 9 October. The evaluation is technical; classroom use so far is informal, and checkout opened to subscribers only on 16 October 2026, so the report makes no claims about learning outcomes or revenue.

CONTENTS

ABSTRACT 2
1. INTRODUCTION 7
1.1 Background 7
1.2 Problem statement 7
1.3 Objectives 7
1.4 Scope and limitations 8
1.5 How this report was produced 8
2. BACKGROUND 9
2.1 Games, anxiety and one shared screen 9
2.2 Managed infrastructure and database-level security 9
2.3 AI coding agents 9
3. SYSTEM DESIGN 10
3.1 Architecture 10
3.2 Data model 11
3.3 Access control 11
3.4 Billing 12
3.5 Privacy by design 13
3.6 Content as data 14
4. THE GAMES 15
4.1 59 Seconds 15
4.2 Taco Trivia 16
4.3 Mango Mafia 17
4.4 Super Mushroom Man 17
4.5 Cookie Code 18
4.6 The platform 19
5. METHOD: DIRECTING AI CODING AGENTS 20
5.1 Who does what 20
5.2 The loop 20
5.3 Rules that made it work 22
5.4 Evidence 22
5.5 What went wrong, and what changed 23
6. EVALUATION 24

6.1 Automated tests and builds 24
6.2 Page performance 24
6.3 Accessibility 26
6.4 Classroom use 26
6.5 Against the objectives 26
7. DISCUSSION AND LIMITATIONS 27
7.1 What the method shows 27
7.2 Limitations 27
8. CONCLUSION AND FUTURE WORK 28
REFERENCES 29
APPENDIX A: PLATFORM DATABASE SUMMARY 31
APPENDIX B: MEASUREMENT METHOD 32
APPENDIX C: LIGHTHOUSE RESULTS 33

LIST OF FIGURES

Figure 1. Class Cookies architecture: six applications, one sign-in, three databases. 10
Figure 2. Entity-relationship diagram of the platform database (25 tables). 11
Figure 3. What a teacher can use, signed out, with a free account, and with a subscription. 12
Figure 4. 59 Seconds: a round in progress with two teams. 16
Figure 5. Taco Trivia: a free pack on the board, three teams. 16
Figure 6. Mango Mafia at night (player names are made up). 17
Figure 7. Super Mushroom Man: the 24-block board and four teams. 18
Figure 8. Cookie Code: two guesses on the board. 18
Figure 9. My Classroom with the made-up class "Mango Test". 19
Figure 10. How the work is done. 21
Figure 11. Commits on each production branch, by who co-wrote them. 23
Figure 12. Automated tests passing, by codebase. 24
Figure 13. Lighthouse Performance score by page, desktop and mobile. 25

LIST OF TABLES

Table 1. The five games, 7 October 2026. 15
Table 2. Tests, checks and builds on fresh clones, 7 October 2026. 24
Table 3. Time to first byte from Seoul, 10 requests per page, 7 October 2026. 26
Table 4. Objectives and evidence. 26
Table A1. The 25 tables of the platform database, by group. 31
Table C1. Lighthouse 13.5.0, median of three runs, 7 October 2026. 33

ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CI	Continuous Integration
CLS	Cumulative Layout Shift
CVD	Colour Vision Deficiency
ESL	English as a Second Language
LCP	Largest Contentful Paint
RLS	Row-Level Security
SSO	Single Sign-On
SUS	System Usability Scale
TBT	Total Blocking Time
TTFB	Time to First Byte
UI	User Interface

1. INTRODUCTION

1.1 Background

I teach English in Seoul. In June 2026 I started 59 Seconds, a speaking game for one classroom projector, as the capstone project of my Master of Science in Information Technology at the University of the People. Its report, submitted in August 2026, set out the idea behind it: students who can speak English often will not, because speaking in front of others is frightening, and most classroom technology makes that worse by putting each student behind a separate screen (Krashen, 1982; Sung et al., 2016).

Over the following weeks I made more games on the same principle: one screen, one teacher at the controls, students facing each other. By September they needed a shared home, so that a teacher could sign in once, keep class lists in one place and use them in any game, and pay for one subscription rather than one purchase per game. That home is Class Cookies, live at classcookies.com since 18 August 2026.

1.2 Problem statement

A teacher who wants a speaking or quiz activity on the classroom screen has two poor choices. Ready-made platforms assume a device per student, which costs time, splits attention and is often impossible in a language school. Home-made slide decks work on one screen but carry no scoring, no randomness and no memory of the class. The problem this project addresses is to give teachers a set of games and tools that run on the equipment they already have, start in seconds, are free to play in full, and keep a class's details safe while letting the teacher reuse them everywhere.

1.3 Objectives

- **O1. Free and immediate.** Every game is playable in full with no sign-in and no student device.
- **O2. One account across separate apps.** Each game is deployed on its own, yet one sign-in and one subscription work in all of them.
- **O3. Class data protected by the database.** Class lists are reusable by tools and games, and the rule for who may read or change them is enforced in the database, not in page code.
- **O4. Content without code changes.** Decks, packs and questions are added as data.

- **O5. Measured quality.** Automated tests, clean builds, and fast pages on the hardware and connections schools have.
- **O6. An honest record of the method.** The report says plainly who wrote what.

1.4 Scope and limitations

The report covers the platform and its five games as they stood on 7 October 2026, when every measurement in it was taken, with two later additions: the real test payment of 8 and 9 October 2026 (Section 3.4), and the opening of checkout to subscribers on 16 October 2026. Three further games are listed on the site as planned; they are not built and are out of scope. Paid subscriptions are built and were tested with three real payments on my own card, all refunded. Checkout opened to subscribers on 16 October 2026, after that test, so this report has no history of paying teachers to evaluate. Classroom use has been informal: I use the games with my own classes most weeks, and colleagues have tried them in theirs. No formal classroom or usability study has been carried out, and the report makes no claim about learning outcomes.

1.5 How this report was produced

Every figure in this report was measured or counted on 7 October 2026: tests and builds on fresh clones of each production branch, page scores and response times on the live sites from Seoul, and commit statistics from Git. The exceptions are the amounts and timings of the test payment in Section 3.4, which come from its record of 8 and 9 October 2026, and the date checkout opened to subscribers, 16 October 2026. Appendix B describes the method. Screens that show a signed-in teacher come from a local test copy of the platform filled with made-up data; no real teacher or student appears anywhere in this report.

2. BACKGROUND

2.1 Games, anxiety and one shared screen

Krashen's (1982) affective filter hypothesis holds that anxiety and low confidence block language acquisition even when learners understand the input. Games lower that filter because attention moves from avoiding mistakes to winning. Wang and Tahir (2020), reviewing 93 studies of Kahoot!, found positive effects on learning, classroom dynamics and anxiety, but also recorded the complaints students make about projected quiz games: text that is hard to read from a seat, and time pressure that feels unfair. Those complaints became design rules for every game here: very large type, timers the teacher controls, and nothing that advances on its own.

Sung et al. (2016), in a meta-analysis of 110 studies, found a moderate positive effect of mobile devices on learning. The platform takes the other path deliberately. For speaking practice the useful interaction is between students, so one shared screen and zero devices is the design, not a compromise.

2.2 Managed infrastructure and database-level security

Jonas et al. (2019) describe serverless computing as removing most system administration from the developer. Class Cookies relies on that: Vercel hosts the applications and Supabase provides the PostgreSQL database, authentication and file storage. The decisive feature is row-level security, which attaches access rules to the table itself (PostgreSQL Global Development Group, n.d.; Supabase, n.d.). Broken access control is the first risk in the OWASP Top 10 (OWASP Foundation, 2021); putting the rule in the database means a page that forgets a check cannot leak or unlock a row.

2.3 AI coding agents

Peng et al. (2023) found, in a controlled experiment, that developers with an AI pair programmer finished a set programming task 55.8% faster than those without. Agentic tools go further: Claude Code reads a codebase, plans changes, edits files and runs commands in the developer's terminal (Anthropic, n.d.). That raises a different question from speed, which Chapter 5 addresses: how a person who does not write the code keeps control of what is built and how it is checked.

3. SYSTEM DESIGN

3.1 Architecture

Class Cookies is six applications on one domain (Figure 1). The platform at classcookies.com is a Next.js 16 application; each game runs on its own subdomain from its own repository and deployment. All six are hosted on Vercel with server functions in Seoul. Keeping the games separate means one can change, or fail, without touching the others.

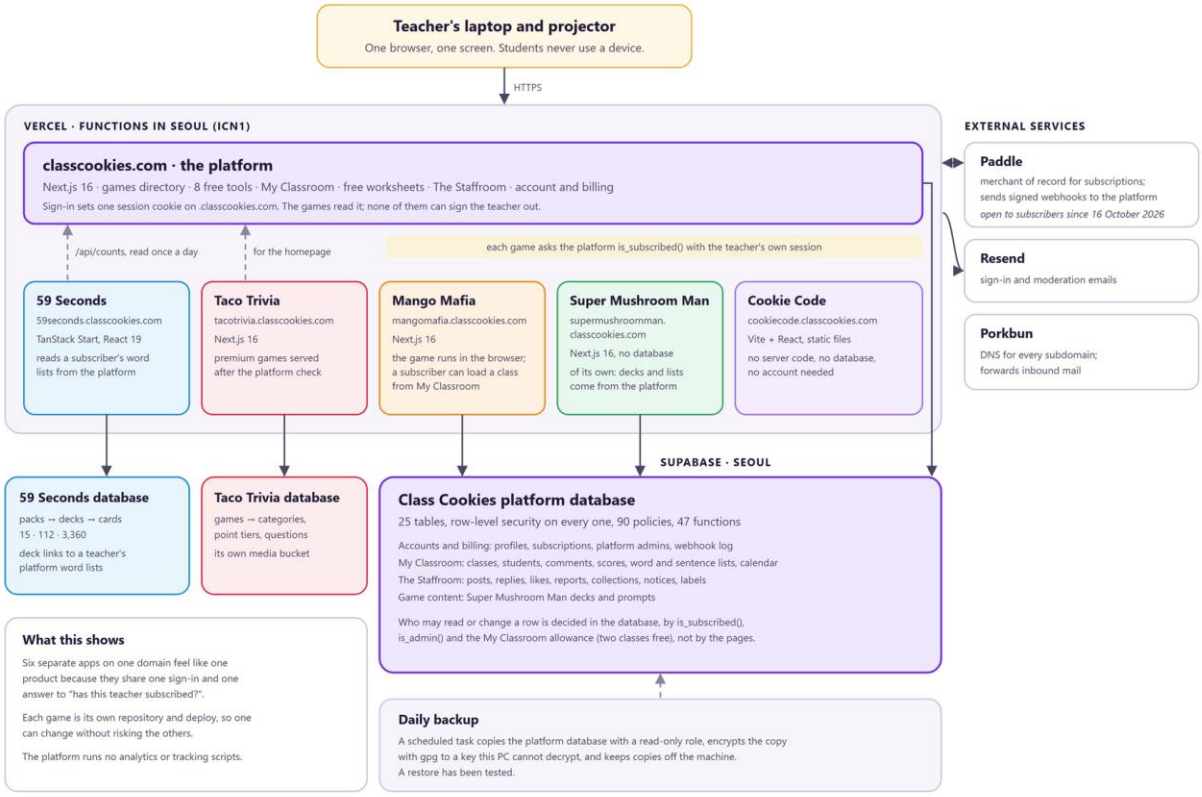


Figure 1. Class Cookies architecture: six applications, one sign-in, three databases.

Two things make the six feel like one product. First, signing in on classcookies.com sets one session cookie scoped to the parent domain, so every subdomain receives it (MDN Web Docs, n.d.); no game can sign the teacher out. Second, every game asks the platform the same question, **is_subscribed()**, using the teacher's own session rather than a privileged key. 59 Seconds and Taco Trivia keep their content in their own databases; Super Mushroom Man reads its decks from the platform; Mango Mafia keeps its game state in the browser; Cookie Code is a static site with no server code at all.

3.2 Data model

The platform database has 25 tables in five groups (Figure 2, Appendix A). Every class, list, calendar note and forum post belongs to a teacher, and deletes cascade down from the teacher, so removing an account removes everything it owns. A class list holds a name, an avatar and an optional gender for each student, and no emails, birthdays or contact details. Comments on students are the most sensitive rows; only the teacher who owns the class can read them.

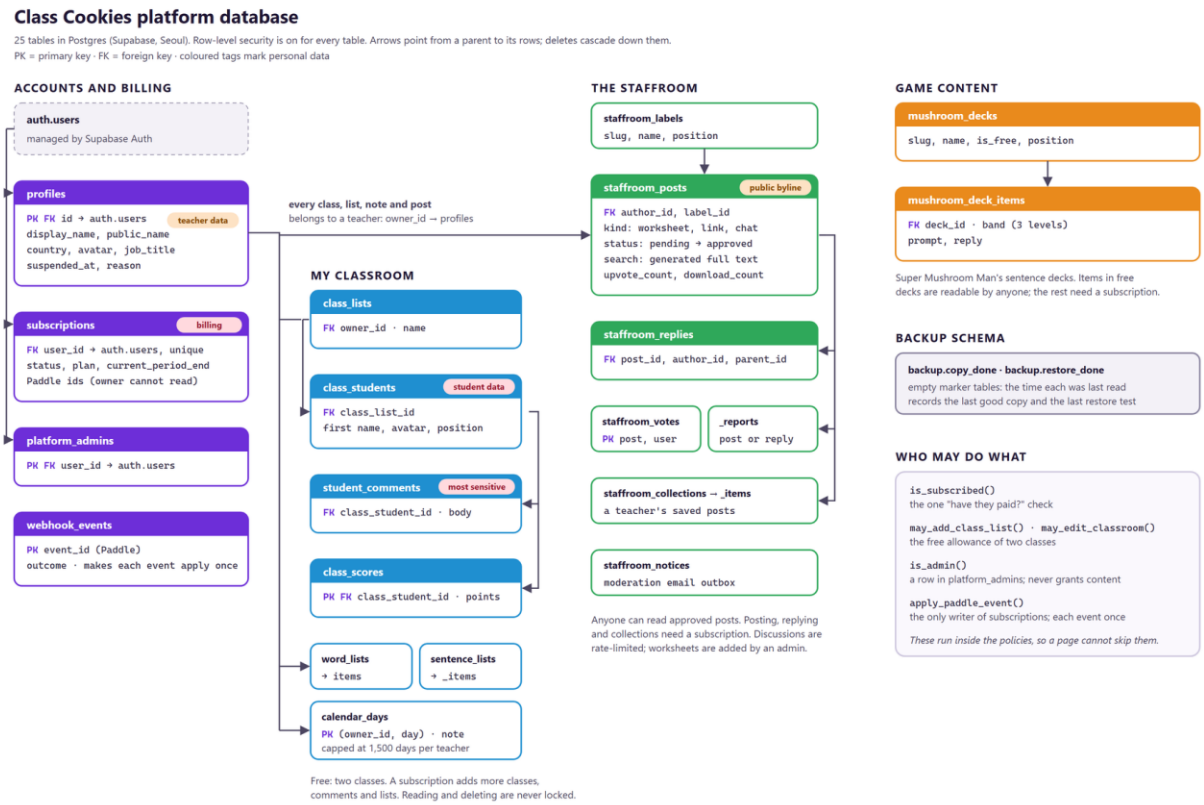


Figure 2. Entity-relationship diagram of the platform database (25 tables).

3.3 Access control

Row-level security is enabled on all 25 tables, with 90 policies. Policies call a small set of functions: **is_subscribed()** is the only test of whether a teacher has paid; **may_add_class_list()** and **may_edit_classroom()** apply the free allowance of two classes; **is_admin()** marks an administrator and never grants access to content. Reading and deleting one's own classes are never locked, so a teacher whose subscription lapses keeps their data. Figure 3 shows what each kind of visitor can use.

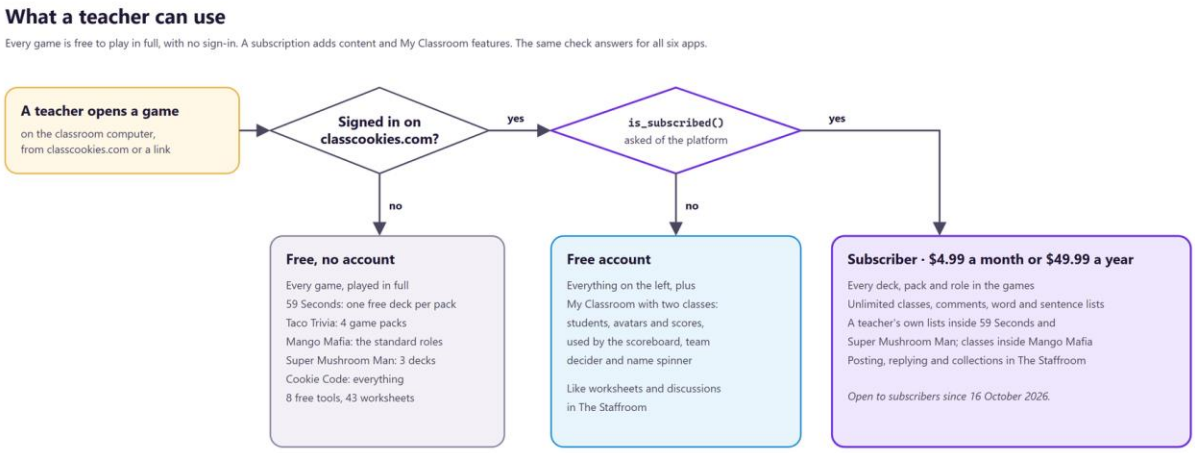


Figure 3. What a teacher can use, signed out, with a free account, and with a subscription.

The privileged service key, which bypasses row-level security, is used in exactly two places: the billing function described next, and the 60-second download links for worksheet files. Administrative work, such as uploading a worksheet, runs under the administrator's own session.

3.4 Billing

Paddle is the merchant of record, so it handles tax and payment details and card data never reaches the platform. Paddle's webhooks are signed, and the platform verifies each signature before trusting the payload (Paddle, n.d.). A single database function applies each event exactly once and is the only code that can write a subscription. Prices are $4.99 a month or $49.99 a year, tax included.

A real test payment. On 8 and 9 October 2026 I paid on my own card, on the live site, with checkout opened to one test account. A $4.99 monthly payment (shown by Paddle as $4.54 plus $0.45 tax) made the account a subscriber within two seconds, and the paid content then opened in Taco Trivia, Mango Mafia, Super Mushroom Man and 59 Seconds, in My Classroom and in The Staffroom. Replaying the same event changed nothing. I then cancelled at the end of the period, cancelled at once and refunded, which removed access; paid a second month; switched to the yearly plan, which charged $45.00 ($49.99 less a $4.99 credit); and cancelled and refunded again. All three payments ($4.99, $4.99 and $45.00) were refunded in full; Paddle's approval of the last $4.99 refund had not yet arrived when this was written, and access was already off.

The test found three faults, each fixed by 9 October. **The API key could only read.** Cancelling failed with a plain "That did not work", because the Paddle key had read permissions and no write

permission, and the controls logged nothing that said why; I gave the key the two write permissions it needs, and each control now logs Paddle's error code and message. **Paddle's refund labels did not mean what the code assumed.** A refund of the whole first payment arrived labelled "partial" at the top level, and the $45.00 refund after the switch was "partial" at every level, because Paddle measures it against the $49.99 list price rather than what was charged. Both were recorded as ignored; access was already off only because the written order cancels before refunding. The rule now is that a refund removes access when it equals what the teacher paid on that payment, which the platform checks by asking Paddle for the payment's total; the $45.00 refund, replayed after the fix, was applied by this rule. **The page flashed after paying.** When Paddle's checkout window closed, the plan buttons showed again for about 200 ms before the page moved on, and the account page, loaded before Paddle's event arrived about two seconds later, said "Ended" and offered a Subscribe button to a teacher who had just paid: a way to pay twice. The page now says "Payment received. You're subscribed." at once, and the account page waits for the subscription, refreshing until it arrives. Two smaller faults were fixed at the same time: the switch's message now states the amount charged that day, and the switch to yearly is no longer offered on a subscription that is set to cancel.

Two parts of the path were not tested live. The email a teacher receives when a renewal payment fails has not been sent to a real address, because Paddle's test tool cannot send that event for a real subscription; its wording and timing are covered by automated tests. Paddle's retry period after a failed payment, and the cancellation at its end, were not tested either. After the test, checkout was closed to the public again until it opened to subscribers on 16 October 2026.

3.5 Privacy by design

- The platform loads no analytics, tag manager or tracking script, and calls no AI service. Automated tests fail if either is added.
- When a teacher opens Mango Mafia with a class, only the class's id travels in the link; the game reads the names with the teacher's own session.
- A daily backup copies the database through a read-only role, encrypts it with gpg to a key the backup machine cannot decrypt, and keeps copies off that machine. A full restore was tested on 2 October 2026.

3.6 Content as data

Content is added without code changes in every game that has a server: 59 Seconds imports decks from spreadsheets through an admin page, Taco Trivia has an admin grid editor, and Super Mushroom Man's decks live in the platform database with their own admin page. Mango Mafia's fourteen roles are part of the game's code, and Cookie Code has no content of its own: the teacher types the word.

4. THE GAMES

Table 1 summarises the five games. Each runs on its own subdomain, is free to play in full, and needs no student device.

Game	What the class does	Content	Tests passing
59 Seconds	One student describes five words; the team guesses in 59 seconds	15 packs, 112 decks, 3,360 cards	111
Taco Trivia	Teams pick questions from a points board	11 game packs, 398 questions	none (no suite)
Mango Mafia	Classroom Mafia, moderated by the teacher	14 roles; 4 to 20 players	729
Super Mushroom Man	Teams say what they would say next; right answers open blocks	3 free decks, 270 prompts	216
Cookie Code	The class cracks the teacher's secret word	3- to 5-letter words, set by the teacher	57

Table 1. The five games, 7 October 2026.

4.1 59 Seconds

A card shows five words; one student describes them while their team guesses before the timer runs out (Figure 4). The timer can be changed with one click, a bell rings once at zero, and the card never moves on its own. Fifteen packs hold 3,360 cards, most of them local to one country, and six packs add a translation line in the local script. In September 2026 the game moved into Class Cookies: its own pack sales ended on 23 September and its payment code was removed on 28 September.

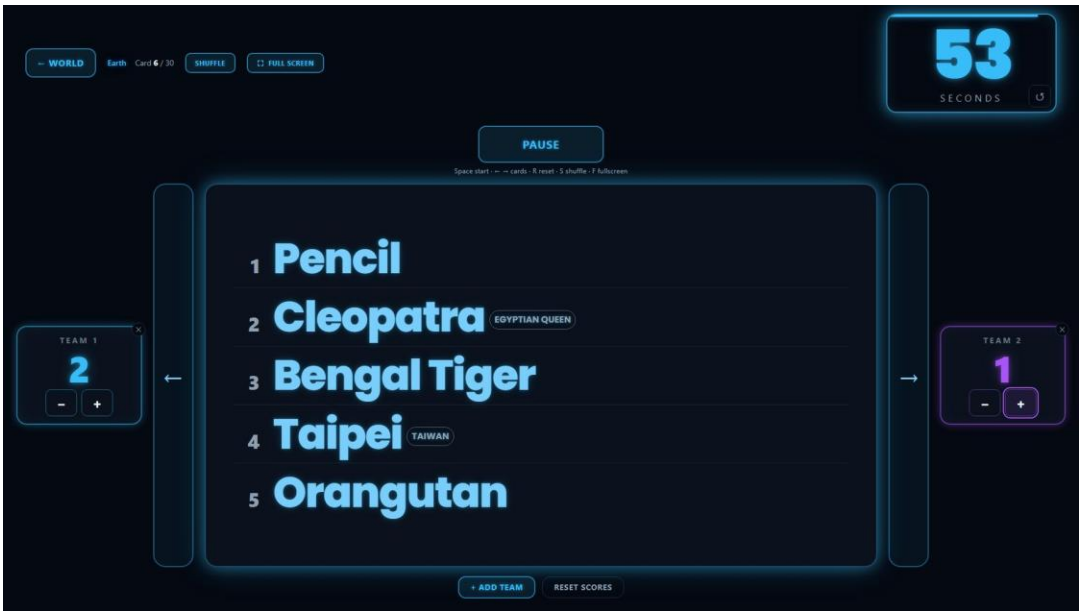


Figure 4. 59 Seconds: a round in progress with two teams.

4.2 Taco Trivia

A Jeopardy-style board of categories and point values for one to six teams (Figure 5). The teacher is the only operator: they open a question, reveal the answer with one key and award points by hand. Four of the eleven game packs are free with no account; premium games are sent only after the platform confirms the subscription. A game in progress survives a page reload.

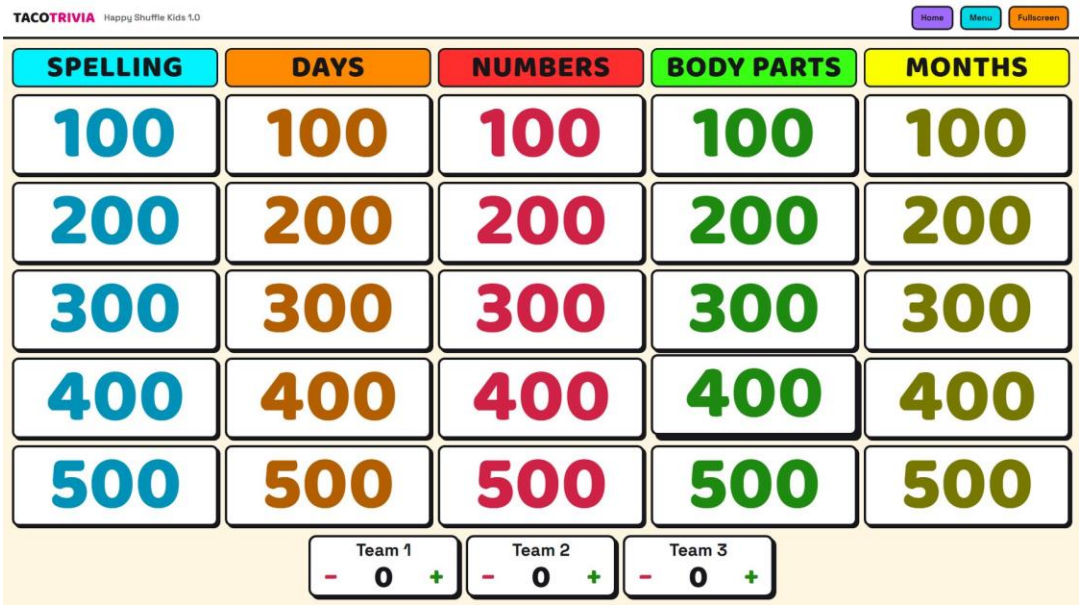


Figure 5. Taco Trivia: a free pack on the board, three teams.

4.3 Mango Mafia

The Mafia party game for four to twenty students, with the teacher as moderator. The app deals the roles, shows each student their role in turn, reads each night-time line for the teacher, and checks for a win after every removal (Figure 6). Its central rule is that the screen is public: nothing selected at night stays visibly marked. Its 729 tests include whole games played to a win and checks that no screen leaks a hidden role.

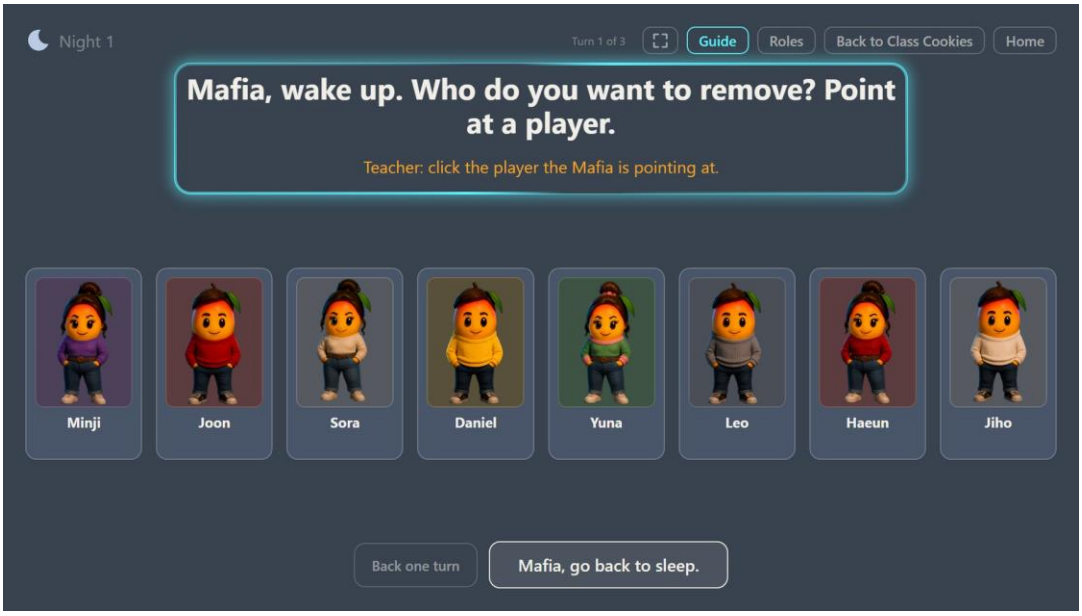


Figure 6. Mango Mafia at night (player names are made up).

4.4 Super Mushroom Man

A situation appears and the team says what they would say next; the teacher judges the answer. A right answer opens a block that pays one to five coins or reveals one of seven mystery outcomes, from doubling a score to swapping it with another team's (Figure 7). Every scoring rule is a small pure function with its own tests, and randomness comes from a seeded generator, so any game can be replayed exactly.

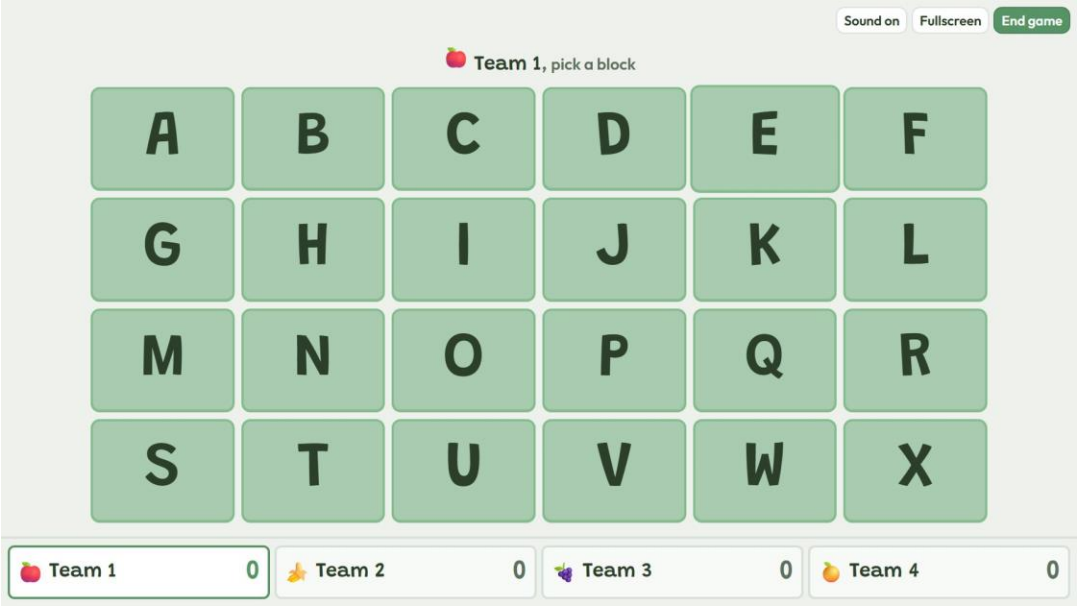


Figure 7. Super Mushroom Man: the 24-block board and four teams.

4.5 Cookie Code

A word-guessing game for the whole class. The teacher types a secret word of three to five letters, hidden as they type; the class calls out guesses and the teacher enters them (Figure 8). It is a static site with no account, no server and no stored data. It is the smallest of the five: eight commits made in one evening, 15 September 2026, from a design prepared beforehand.

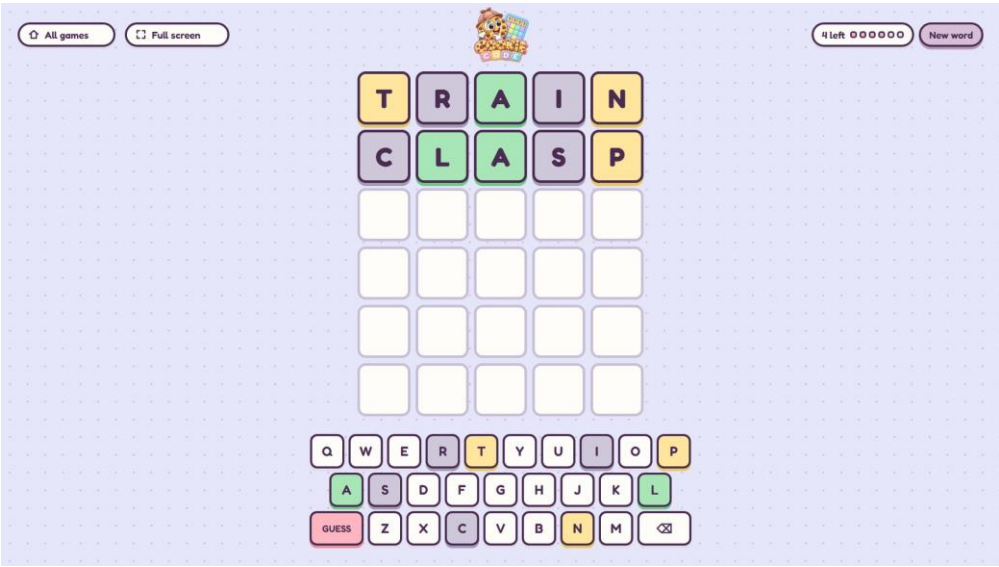


Figure 8. Cookie Code: two guesses on the board.

4.6 The platform

Around the games, the platform offers eight free tools (timer, stopwatch, clock, a name spinner, team decider, coin flip, scoreboard and whiteboard), My Classroom for class lists, scores, comments and word and sentence lists (Figure 9), 43 free worksheets, and The Staffroom, a moderated forum where subscribed teachers post and reply.

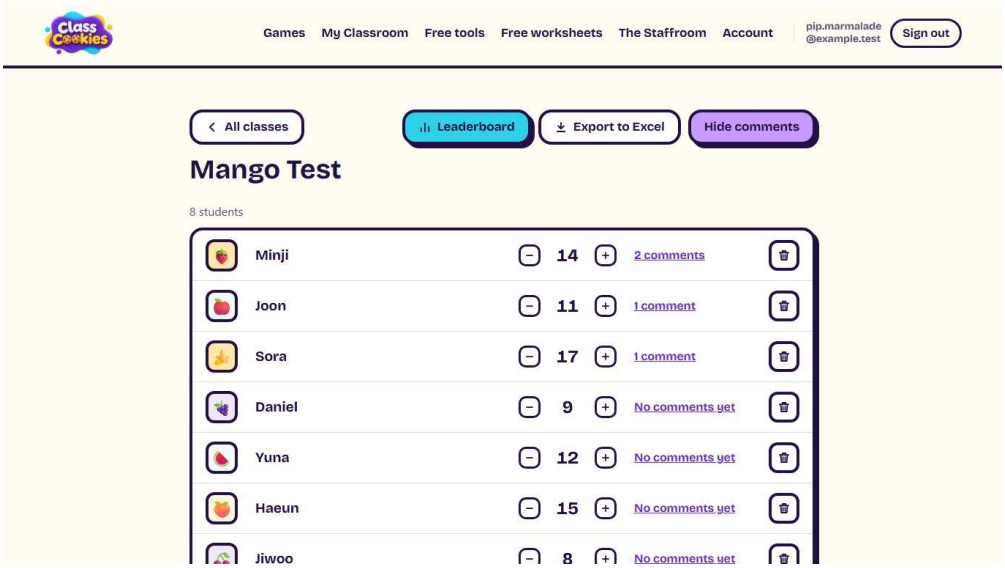


Figure 9. My Classroom with the made-up class "Mango Test".

5. METHOD: DIRECTING AI CODING AGENTS

5.1 Who does what

I am the product owner and designer. I decide what to build and why, design how it works in a classroom, write the rules and acceptance criteria, approve each plan, and decide every release. The code is written by AI coding agents. 59 Seconds began in June 2026 as a scaffold generated by Lovable; since then Claude Code has written the code for all six products, and two chat-based review agents check the same work. Git lists me as the author of most commits because I direct the agents and commit their work; it does not mean I typed the code. The products themselves contain no AI.

5.2 The loop

Each change follows the loop in Figure 10. The brief and the rules live in an instruction file in each repository, which every agent session reads first, and in task briefs for larger pieces of work.

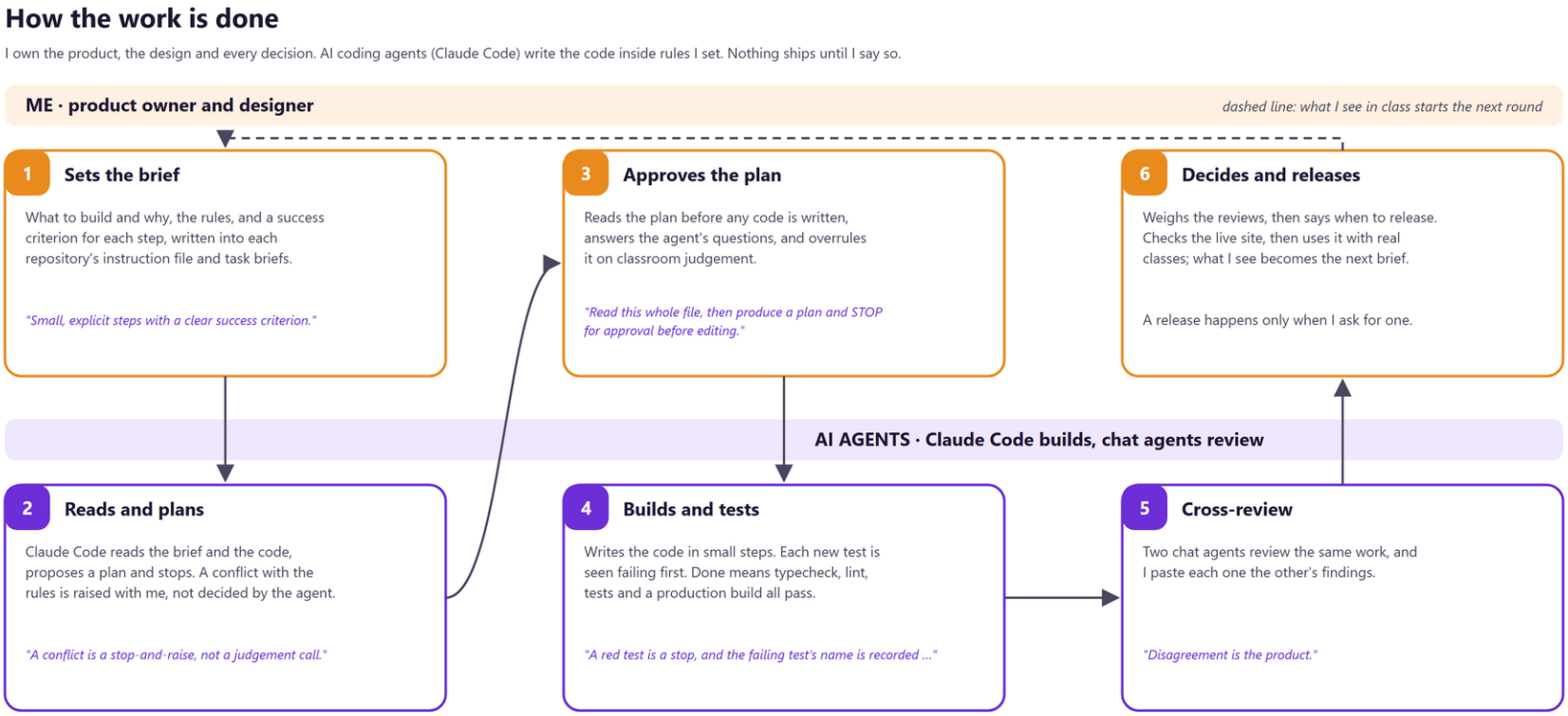


Figure 10. How the work is done.

5.3 Rules that made it work

A handful of rules, written into the instruction files, did most of the work of keeping the agents on course. They are quoted here as written.

- **Plan first.** "Read this whole file, then produce a plan and STOP for approval before editing." No code is written until I have read the plan.
- **Small steps.** "Small, explicit steps with a clear success criterion."
- **Conflicts come to me.** "A conflict is a stop-and-raise, not a judgement call." When the brief and the code disagree, the agent asks rather than chooses.
- **Tests are evidence only if they can fail.** New tests are seen failing first, and "A red test is a stop, and the failing test's name is recorded …"
- **Done means checked.** Typecheck, lint, tests and a production build all pass before a change is called done.
- **I release.** A release happens only when I ask for one, and no agent changes the hosting, the domain records or the database dashboard without a fresh yes from me.
- **Two reviewers.** Two chat agents review the same work, and I pass each one the other's findings. As the brief puts it, "Disagreement is the product."

5.4 Evidence

Commits written with Claude Code carry a co-author line. Figure 11 counts them on each production branch: 694 of 869 commits (80%). The 159 commits by the Lovable bot are the generated scaffold that 59 Seconds started from.

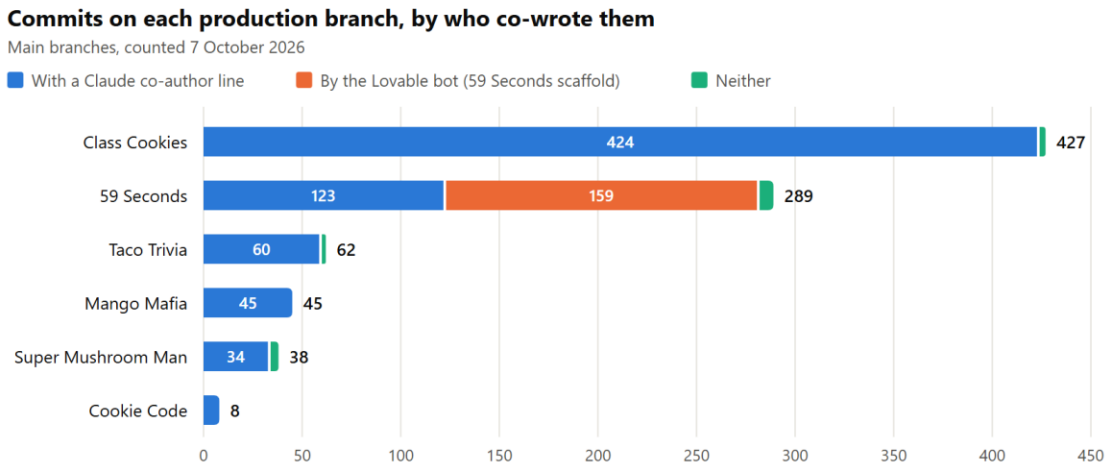


Figure 11. Commits on each production branch, by who co-wrote them.

5.5 What went wrong, and what changed

A refund that did not remove access. 59 Seconds went live with real payments on 17 July 2026. The only live purchase was my own test, and it was refunded, but the refund did not remove access, because the webhook was not yet listening for refund events; that access was removed on 16 September 2026. My capstone report said the refund had revoked access, and the first version of my portfolio claimed "real payments, verified refunds"; both were wrong, and both now carry a dated correction. The lesson I took is that a test of the happy path is not a test of the whole flow.

A silent bell. In August, system tests against the live 59 Seconds site found that the end-of-round bell never rang, although all 24 unit tests passed. The fix pulled the timer logic into two small functions and added a regression test that plays whole rounds at six lengths and expects exactly one ring.

Documentation that ran ahead of the system. The platform's own records once described a redirect as working before it existed; the notes record that the claim itself was why nobody checked. This report applies the lesson: every figure in it was checked against the live sites or the code on the day it was measured.

6. EVALUATION

6.1 Automated tests and builds

Each production branch was cloned fresh and its checks run on 7 October 2026 (Table 2, Figure 12). In all, 3,783 tests passed and none failed. Taco Trivia has no automated test suite; its changes are checked with scripted browser runs and before-and-after screenshots. Its build needs production settings that were not copied into the clone, but the same commit built successfully on its host.

Codebase	Test files	Tests passed	Typecheck	Lint / build
Class Cookies	98	2,670	0 errors	0 errors / passes
59 Seconds	11	111	0 errors	12 errors (its recorded baseline) / passes
Taco Trivia	none	none	in build	0 errors / passes on host
Mango Mafia	57	729	in build	0 problems / passes
Super Mushroom Man	10	216	0 errors	0 problems / passes
Cookie Code	3	57	in build	0 problems / passes

Table 2. Tests, checks and builds on fresh clones, 7 October 2026.

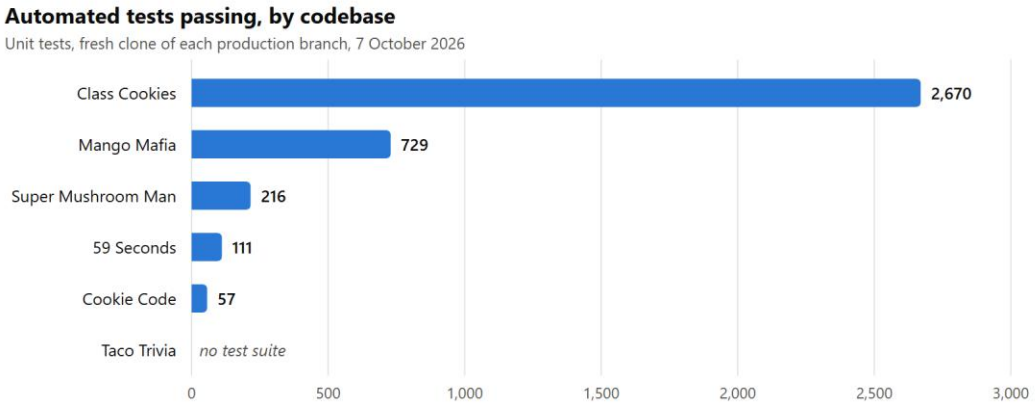


Figure 12. Automated tests passing, by codebase.

6.2 Page performance

Eleven public pages were measured with Lighthouse 13.5.0 (Google, n.d.) under its desktop and mobile presets, three runs each (Figure 13; full results in Appendix C). Seven pages scored 93 or more on both. Largest Contentful Paint, which web.dev counts as good at 2.5 seconds or less (Walton, n.d.), was at most 1.35 seconds on desktop for every page except the 59 Seconds home page.

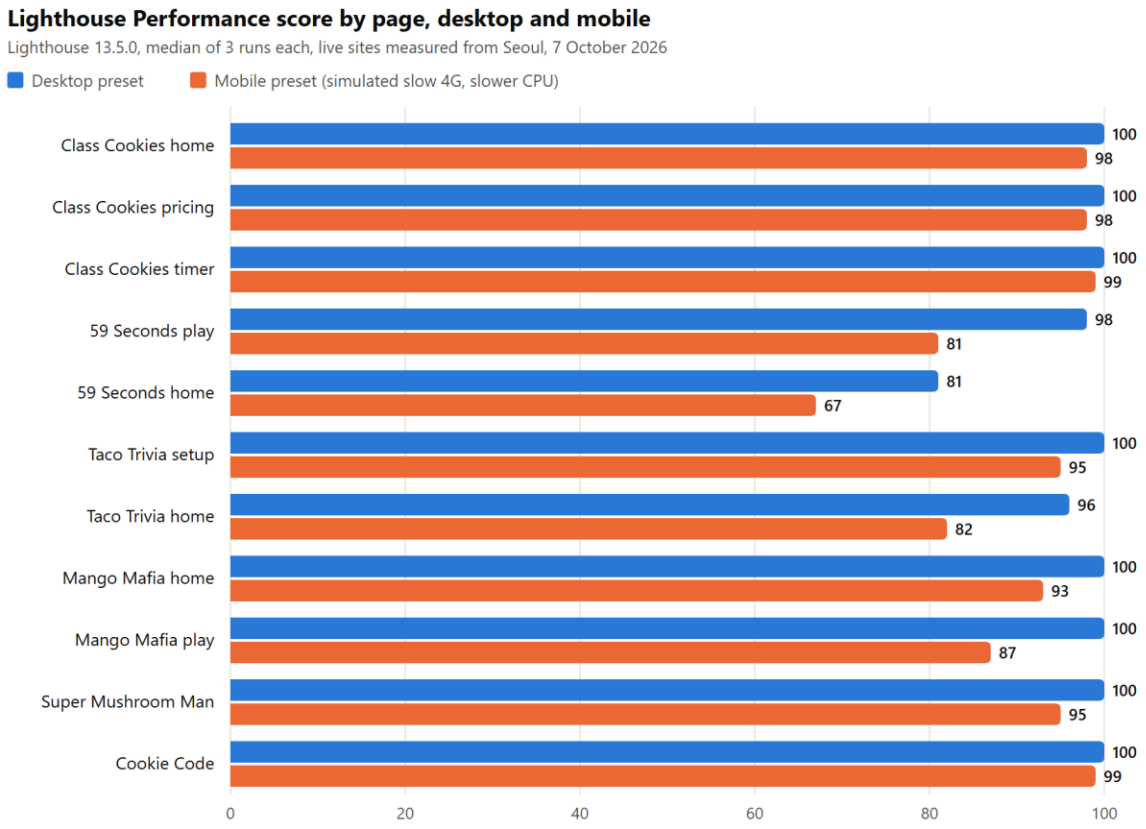


Figure 13. Lighthouse Performance score by page, desktop and mobile.

Three of the four weaker pages share one cause: images. 59 Seconds' home page loaded about 2.4 MB of images, two of them about 1.1 MB each; Mango Mafia's play screen loads 2.5 MB of character art; Taco Trivia's home loads a cover for every game. The fourth, 59 Seconds' play screen, weighs only 243 kB but scores 81 on mobile because its web fonts and stylesheet block the first paint: Lighthouse estimates 1.3 seconds of savings there. Scripts are 140 to 185 kB on every page. The capstone reported a desktop score of 96 for 59 Seconds in August, on a different home page; it was 81 on 7 October, while the play screen that a class actually sees scored 98. Later on 7 October, after these measurements, the home page's pack covers were replaced with resized copies, and the project's own notes record its page weight falling from 2,653 KB to 361 KB and its score rising to 95 on desktop and 78 on mobile (Lighthouse 12.8.2, median of three runs, so not directly comparable with Figure 13).

Page	Median	p90	Served from
classcookies.com home	58 ms	68 ms	cache
classcookies.com pricing	52 ms	59 ms	cache

Page	Median	p90	Served from
59 Seconds home	109 ms	145 ms	server
59 Seconds play	135 ms	149 ms	server
Taco Trivia home	157 ms	174 ms	cache
Taco Trivia setup	174 ms	212 ms	server
Mango Mafia home	55 ms	67 ms	cache
Super Mushroom Man	51 ms	64 ms	cache
Cookie Code	55 ms	71 ms	cache

Table 3. Time to first byte from Seoul, 10 requests per page, 7 October 2026.

6.3 Accessibility

Every measured page scored 92 to 100 for Lighthouse Accessibility. That means no automated check failed; it does not mean the pages conform to WCAG 2.2 (World Wide Web Consortium [W3C], 2024), which needs testing with assistive technology and with users. The capstone found the gap in practice: a page that scored 100 was still unreadable to children sitting at the back of a room.

6.4 Classroom use

I use the games with my own classes most weeks, and colleagues have tried them in theirs. This is everyday, informal use, not a study: there was no protocol, no measurement and no comparison, so it supports no claim about learning.

6.5 Against the objectives

Objective	Evidence	Status
O1 Free and immediate	All five games play in full signed out; no student device anywhere	Met
O2 One account across apps	Shared cookie and is_subscribed(); a real payment unlocked every paid area (Section 3.4)	Met; now open to subscribers
O3 Data protected by the database	RLS on 25 of 25 tables, 90 policies; service key used in two places	Met
O4 Content without code	Admin import or editors in the three games with servers	Met for content games
O5 Measured quality	3,783 tests passing; 7 of 11 pages at 93 or more on both presets	Partly met: Taco Trivia has no tests; four slower pages on mobile
O6 Honest record	Chapter 5; the corrected refund claim	Met

Table 4. Objectives and evidence.

7. DISCUSSION AND LIMITATIONS

7.1 What the method shows

One teacher, without writing code, directed the building of six live applications in about four months, with tests, database-level security and a record of every decision. The method's strength is that the rules are written down and checked: an agent that is told to stop when a test is red, and to ask when the brief and the code disagree, makes fewer silent mistakes. Its weakness showed in the refund: the checks I set tested what I thought to test, and the missing case was mine.

7.2 Limitations

- **No automated pipeline.** The agents run typecheck, lint, tests and build before each release, but nothing enforces it between a commit and production (Red Hat, 2025).
- **Taco Trivia has no test suite.**
- **One payment test before opening.** The billing path had been through three real payments, made and refunded on my own card on 8 and 9 October 2026 (Section 3.4), when checkout opened to subscribers on 16 October 2026. This report therefore has no history of paying teachers to evaluate. The failed-payment email to a teacher and Paddle's retry period have not been tested live.
- **Page weight.** On 7 October large images slowed three pages on mobile (59 Seconds' home page has since been fixed), and render-blocking fonts a fourth. Some deck and question images come from stock-photo sites or have no recorded source, and a rights review of content images is outstanding.
- **One maintainer and single providers.** If I am unavailable, nothing ships; a change at Vercel, Supabase or Paddle lands directly on the product.
- **Informal evaluation.** No usability study and no classroom study; performance measured from one city, with simulated throttling.

8. CONCLUSION AND FUTURE WORK

Class Cookies meets most of what it set out to do: five games that a teacher can start in seconds on one screen, one account across separately deployed apps, class data protected in the database, and quality that is measured rather than claimed. It also records, with its mistakes, how a non-programmer can direct AI coding agents to build software that is real enough to measure.

- **Next technical steps:** an automated check on every change before it reaches production; a test suite for Taco Trivia; smaller images and non-blocking fonts on the four slower pages; the rights review of content images.
- **Next product step:** checkout opened to subscribers on 16 October 2026, after a real test payment on 8 and 9 October 2026 found three faults, all fixed. The next step is measuring the billing path with real use, including the failed-payment email and Paddle's retry period, which that test could not reach.
- **Next evaluation step:** a short task-based usability study with five or six teachers using the System Usability Scale (Bangor et al., 2008), and later a classroom study of how much students actually speak.

REFERENCES

Anthropic. (n.d.). Claude Code overview. Claude Code Docs. Retrieved October 7, 2026, from https://code.claude.com/docs/en/overview

Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the System Usability Scale. International Journal of Human-Computer Interaction, 24(6), 574-594. https://doi.org/10.1080/10447310802205776

Google. (n.d.). Introduction to Lighthouse. Chrome for Developers. Retrieved October 7, 2026, from https://developer.chrome.com/docs/lighthouse/overview

Jonas, E., Schleier-Smith, J., Sreekanti, V., Tsai, C.-C., Khandelwal, A., Pu, Q., Shankar, V., Carreira, J., Krauth, K., Yadwadkar, N., Gonzalez, J. E., Popa, R. A., Stoica, I., & Patterson, D. A. (2019). Cloud programming simplified: A Berkeley view on serverless computing (arXiv:1902.03383). arXiv. https://arxiv.org/abs/1902.03383

Krashen, S. D. (1982). Principles and practice in second language acquisition. Pergamon Press. http://www.sdkrashen.com/content/books/principles_and_practice.pdf

MDN Web Docs. (n.d.). Set-Cookie header. Mozilla. Retrieved October 7, 2026, from https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie

OWASP Foundation. (2021). A01:2021 Broken access control. OWASP Top 10:2021. https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/

Paddle. (n.d.). Verify webhook signatures. Paddle Developer Docs. Retrieved October 7, 2026, from https://developer.paddle.com/webhooks/about/signature-verification/

Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot (arXiv:2302.06590). arXiv. https://arxiv.org/abs/2302.06590

PostgreSQL Global Development Group. (n.d.). Row security policies. PostgreSQL 18 documentation. Retrieved October 7, 2026, from https://www.postgresql.org/docs/current/ddl-rowsecurity.html

Red Hat. (2025, April 29). What is DevOps? https://www.redhat.com/en/topics/devops/what-is-devops

Supabase. (n.d.). Row level security. Supabase Docs. Retrieved October 7, 2026, from https://supabase.com/docs/guides/database/postgres/row-level-security

Sung, Y.-T., Chang, K.-E., & Liu, T.-C. (2016). The effects of integrating mobile devices with teaching and learning on students' learning performance: A meta-analysis and research synthesis. Computers & Education, 94, 252-275. https://doi.org/10.1016/j.compedu.2015.11.008

Walton, P. (n.d.). Web Vitals. web.dev. Retrieved October 7, 2026, from https://web.dev/articles/vitals

Wang, A. I., & Tahir, R. (2020). The effect of using Kahoot! for learning: A literature review. Computers & Education, 149, Article 103818. https://doi.org/10.1016/j.compedu.2020.103818

World Wide Web Consortium. (2024, December 12). Web Content Accessibility Guidelines (WCAG) 2.2 (W3C Recommendation). https://www.w3.org/TR/WCAG22/

APPENDIX A: PLATFORM DATABASE SUMMARY

Group	Tables	Personal data
Accounts and billing	profiles, subscriptions, platform_admins, webhook_events	Teacher details; billing identifiers
My Classroom	class_lists, class_students, student_comments, class_scores, word_lists, word_list_items, sentence_lists, sentence_list_items, calendar_days	Student names and comments; teacher notes
The Staffroom	staffroom_labels, staffroom_posts, staffroom_replies, staffroom_votes, staffroom_reports, staffroom_collections, staffroom_collection_items, staffroom_notices	Public bylines; reports
Game content	mushroom_decks, mushroom_deck_items	None
Backup	backup.copy_done, backup.restore_done (marker tables)	None

Table A1. The 25 tables of the platform database, by group.

Row-level security is enabled on every table, with 90 policies across them; 47 database functions include the access functions named in Section 3.3. Counts are from the platform's migration files on its production branch.

APPENDIX B: MEASUREMENT METHOD

- **Code.** Each production branch was cloned fresh from GitHub on 7 October 2026 and installed with npm ci --ignore-scripts. Tests ran with Vitest's JSON reporter; Class Cookies' 180 integration tests need a local database and were not run.
- **Builds.** npm run build with no environment variables. Taco Trivia's build needs its production settings; its host's production build of the same commit is used instead.
- **History.** Commits on each main branch, counting messages with a Claude co-author line and commits by the Lovable bot.
- **Pages.** Lighthouse 13.5.0 in a separate copy of Chromium, desktop and mobile presets, three runs per page, median reported, run from Seoul.
- **Response times.** Ten requests per page, three seconds apart, each on a new connection. An earlier, faster attempt triggered the host's bot protection; those results were discarded.
- **Content counts.** The public count endpoints of 59 Seconds and Taco Trivia, and Super Mushroom Man's signed-out setup endpoint.

APPENDIX C: LIGHTHOUSE RESULTS

Page	Perf. desktop	Perf. mobile	A11y	SEO	LCP desktop	LCP mobile
Class Cookies home	100	98	96	100	0.44 s	2.11 s
Class Cookies pricing	100	98	100	100	0.46 s	2.13 s
Class Cookies timer	100	99	100	100	0.46 s	2.05 s
59 Seconds home	81	67	100	100	3.27 s	5.56 s
59 Seconds play	98	81	100	100	1.04 s	3.99 s
Taco Trivia home	96	82	98	100	1.35 s	5.03 s
Taco Trivia setup	100	95	100	100	0.53 s	2.71 s
Mango Mafia home	100	93	100	100	0.51 s	3.11 s
Mango Mafia play	100	87	100	100	0.60 s	3.96 s
Super Mushroom Man	100	95	96 / 92	100	0.53 s	2.57 s
Cookie Code	100	99	100	91	0.41 s	1.70 s

Table C1. Lighthouse 13.5.0, median of three runs, 7 October 2026.

Accessibility and SEO are shown for the desktop preset; they were the same on mobile except Super Mushroom Man's accessibility (96 desktop, 92 mobile).